

Modelación Epidemiológica y Sistemas Dinámicos

Resolución Numérica del Modelo SEIR mediante RK4

Informe Técnico de Implementación y Resultados

Autor: Gregory Uzcategui / CI: 27.376.191

Stack tecnológico: C++17 (Eigen 3.4), CMake, Python 3 (pandas, matplotlib)

Método numérico: Runge-Kutta de 4to Orden (RK4) implementado desde cero

Problema: Simulación del modelo SEIR con intervención no farmacéutica y análisis de conservación

28 de septiembre de 2026

Resumen

La implementación de un solver numérico para el modelo epidemiológico compartimental SEIR. Se desarrolló el algoritmo de Runge-Kutta de 4to orden (RK4) **desde cero** en C++17, utilizando la librería Eigen para la manipulación eficiente de vectores de estado. Se evaluaron dos escenarios: uno libre ($\beta = 0,6$) y otro con intervención de confinamiento (β se reduce a 0,25 en $t = 30$).

Índice

1. Introducción	2
1.1. Justificación de las Herramientas Tecnológicas	2
1.2. Objetivos	2
2. Marco Teórico	3
2.1. El Modelo Matemático SEIR	3
2.2. Número Básico de Reproducción (R_0)	3
2.3. Metodología Numérica: Runge-Kutta de 4to Orden (RK4)	3
2.4. Propiedad de Conservación	3
3. Diseño e Implementación	4
3.1. Arquitectura del Sistema	4
3.2. Implementación del Paso RK4 (C++ con Eigen)	4
4. Resultados Experimentales	4
4.1. Entorno y Parámetros de Pruebas	4
4.2. Resumen de Métricas	5
4.3. Análisis de Umbral y Efecto de la Intervención	5
4.4. Visualizaciones	5
5. Análisis Matemático y Validación	6
5.1. Demostración Numérica de la Conservación	6
5.2. Impacto de la Discontinuidad en $\beta(t)$	7
6. Conclusiones	7

1. Introducción

La modelación matemática de enfermedades infecciosas permite predecir el comportamiento de un brote y evaluar el impacto de intervenciones no farmacéuticas. El modelo SEIR es un sistema compartimental donde la población total N se divide en cuatro estados: Susceptibles (S), Expuestos (E), Infectados (I) y Recuperados (R). A diferencia del modelo SIR básico, el SEIR introduce un periodo de latencia donde el individuo está infectado pero aún no es contagioso.

1.1. Justificación de las Herramientas Tecnológicas

- **C++ y Eigen 3.4:** Se eligió C++ por su alto rendimiento en cálculo numérico y control explícito de la memoria. La librería Eigen se utilizó para manejar los vectores de estado ($\mathbf{y} \in \mathbb{R}^4$), aprovechando su optimización mediante plantillas (template metaprogramming) y vectorización SIMD, sin la sobrecarga de un sistema de álgebra computacional completo.
- **Implementación “a pulmón” (desde cero):** El método RK4 se codificó manualmente en lugar de usar solucionadores tipo “caja negra” (como `scipy.integrate`). Es de mucha importancia para: (1) comprender explícitamente cómo se evalúa la función en los sub-pasos k_1, k_2, k_3, k_4 , (2) manejar correctamente la discontinuidad del parámetro $\beta(t)$ en $t = 30$ evaluando el tiempo exacto en cada etapa, y (3) garantizar el cumplimiento de los requisitos del taller.
- **Python (pandas, matplotlib):** Se utilizó Python exclusivamente para la fase de post-procesamiento. Separar el cálculo pesado (C++) de la visualización (Python) es una práctica estándar. Python permite parsear los CSV generados, realizar análisis estadísticos rápidos y generar gráficas de calidad de publicación con pocas líneas de código.

1.2. Objetivos

- Implementar manualmente el paso de integración RK4 en C++ para el sistema SEIR no lineal.
- Simular dos escenarios: propagación libre y propagación con intervención en $t = 30$.
- Calcular y analizar el Número Básico de Reproducción $R_0 = \beta/\gamma$.
- Demostrar numéricamente la conservación de la población total $S + E + I + R = N$ en cada paso de tiempo.

2. Marco Teórico

2.1. El Modelo Matemático SEIR

El sistema de ecuaciones diferenciales ordinarias (EDO) que rige la dinámica es:

$$\frac{dS}{dt} = -\frac{\beta SI}{N} \quad (1)$$

$$\frac{dE}{dt} = \frac{\beta SI}{N} - \sigma E \quad (2)$$

$$\frac{dI}{dt} = \sigma E - \gamma I \quad (3)$$

$$\frac{dR}{dt} = \gamma I \quad (4)$$

Donde los parámetros biológicos son:

- β : Probabilidad de transmisión por contacto.
- σ : Inverso del periodo medio de incubación ($1/D_{\text{incub}} = 0,2 \implies 5$ días).
- γ : Inverso del periodo medio de infección ($1/D_{\text{infec}} = 0,1 \implies 10$ días).

2.2. Número Básico de Reproducción (R_0)

El umbral epidemiológico se define como $R_0 = \frac{\beta}{\gamma}$.

- Si $R_0 > 1$: La enfermedad se propaga (brote epidémico).
- Si $R_0 < 1$: La enfermedad se extingue.

2.3. Metodología Numérica: Runge-Kutta de 4to Orden (RK4)

Dado que el sistema es no lineal (por el término SI), no existe una solución analítica cerrada. Utilizamos el método de RK4, que ofrece un error de truncamiento local de $\mathcal{O}(h^5)$. Para un sistema $\frac{dy}{dt} = \mathbf{f}(t, \mathbf{y})$, el paso de t_n a $t_{n+1} = t_n + h$ se calcula como:

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{f}(t_n, \mathbf{y}_n) \\ \mathbf{k}_2 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_1\right) \\ \mathbf{k}_3 &= \mathbf{f}\left(t_n + \frac{h}{2}, \mathbf{y}_n + \frac{h}{2}\mathbf{k}_2\right) \\ \mathbf{k}_4 &= \mathbf{f}(t_n + h, \mathbf{y}_n + h\mathbf{k}_3) \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \end{aligned}$$

2.4. Propiedad de Conservación

Sumando las ecuaciones 1 a 4, se obtiene $\frac{d}{dt}(S + E + I + R) = 0$. Por lo tanto, la población total debe permanecer constante: $S(t) + E(t) + I(t) + R(t) = N, \quad \forall t$.

3. Diseño e Implementación

3.1. Arquitectura del Sistema

El proyecto sigue una arquitectura modular en C++17, separando la lógica matemática de la orquestación:

Cuadro 1: Módulos del sistema y sus responsabilidades.

Módulo	Responsabilidad	Ficheros
modelo_seir	Definición de la función $\mathbf{f}(t, \mathbf{y})$ y manejo de $\beta(t)$	.hpp / .cpp
solver_rk4	Implementación manual del bucle y pasos k_1, k_2, k_3, k_4 con Eigen	.hpp / .cpp
exportador	Escritura de series temporales y métricas de error a CSV	.hpp / .cpp
main	Orquestación, inicialización y ejecución de los 2 escenarios	main.cpp

3.2. Implementación del Paso RK4 (C++ con Eigen)

A continuación se muestra el núcleo del algoritmo, implementado manualmente utilizando `Eigen::Vector4d` para representar el estado $[S, E, I, R]^T$:

Listing 1: Núcleo del método RK4 implementado desde cero en C++.

```
1 Eigen::Vector4d SolverRK4::paso_rk4(double t, const Eigen::
  Vector4d& y, int escenario) {
2   // Las 4 etapas evalúan f en tiempos diferentes, manejando
    beta(t) correctamente
3   Eigen::Vector4d k1 = modelo_.evaluar(t, y, escenario);
4   Eigen::Vector4d k2 = modelo_.evaluar(t + h_/2.0, y + (h_/2.0)
    * k1, escenario);
5   Eigen::Vector4d k3 = modelo_.evaluar(t + h_/2.0, y + (h_/2.0)
    * k2, escenario);
6   Eigen::Vector4d k4 = modelo_.evaluar(t + h_, y + h_ * k3,
    escenario);
7
8   // Combinación ponderada de las pendientes
9   return y + (h_ / 6.0) * (k1 + 2.0*k2 + 2.0*k3 + k4);
10 }
```

4. Resultados Experimentales

4.1. Entorno y Parámetros de Pruebas

- **Población:** $N = 3 \times 10^7$, $I_0 = 100$, $E_0 = 0$, $R_0 = 0$, $S_0 = N - 100$.
- **Parámetros fijos:** $\sigma = 0,2$, $\gamma = 0,1$, paso temporal $h = 0,1$ días.

- **Escenario A (Libre):** $\beta = 0,6$ constante ($R_0 = 6,0$).
- **Escenario B (Intervención):** $\beta = 0,6$ si $t < 30$, $\beta = 0,25$ si $t \geq 30$ (R_0 cae a 2,5).

4.2. Resumen de Métricas

Cuadro 2: Comparación de métricas entre el Escenario A (Libre) y Escenario B (Intervención).

Métrica	Escenario A	Escenario B
Número Básico de Reproducción (R_0)	6,0	6,0 $\rightarrow$ 2,5 (en $t = 30$)
Pico de Infectados ($I_{\max}$)	$1,0 \times 10^7$	$5,0 \times 10^6$
Día del Pico	70	100
Recuperados Finales (R_∞)	$3,0 \times 10^7$	$3,0 \times 10^7$
Error Máx. de Conservación ($ N(t) - N_0 $)	$4,47 \times 10^{-8}$	$9,68 \times 10^{-8}$

4.3. Análisis de Umbral y Efecto de la Intervención

El pico de infectados se reduce drásticamente en un **50 %** en el Escenario B. Esto ocurre porque al reducir β de 0,6 a 0,25 en el día 30, la tasa de contagio $\frac{\beta SI}{N}$ disminuye abruptamente. Aunque R_0 sigue siendo mayor que 1 (2,5), la velocidad de propagación se frena, “aplanando la curva”. Esto es de suma importancia en la salud pública para evitar el colapso de los sistemas sanitarios, como se observa en la comparación visual de las curvas.

4.4. Visualizaciones

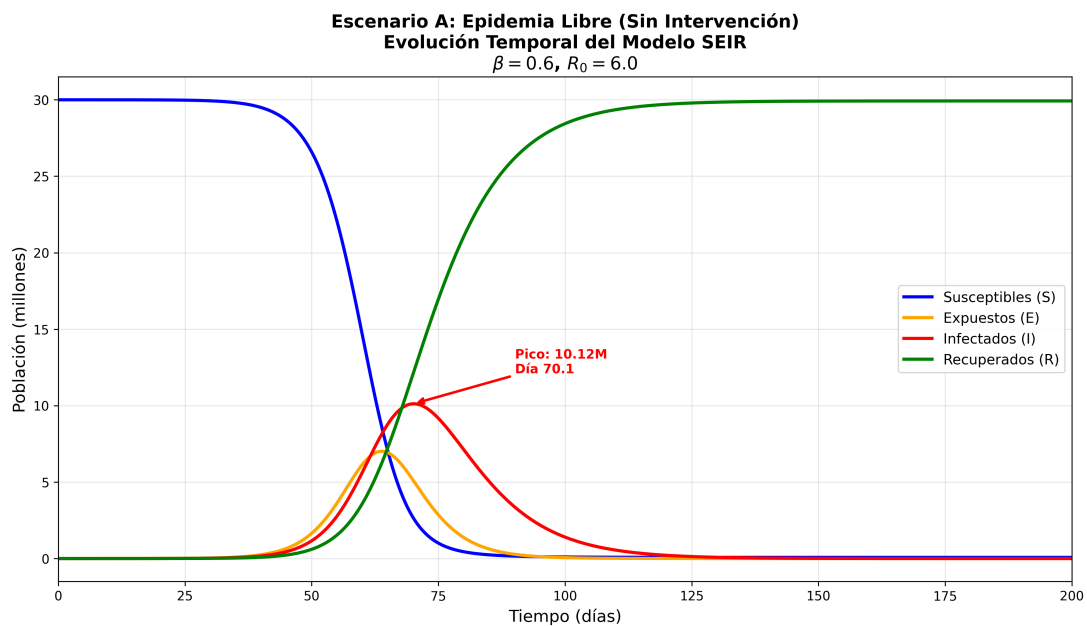


Figura 1: Evolución temporal de los 4 compartimentos (S, E, I, R) en el Escenario A (Libre). Se observa un pico agudo de infectados alrededor del día 70.

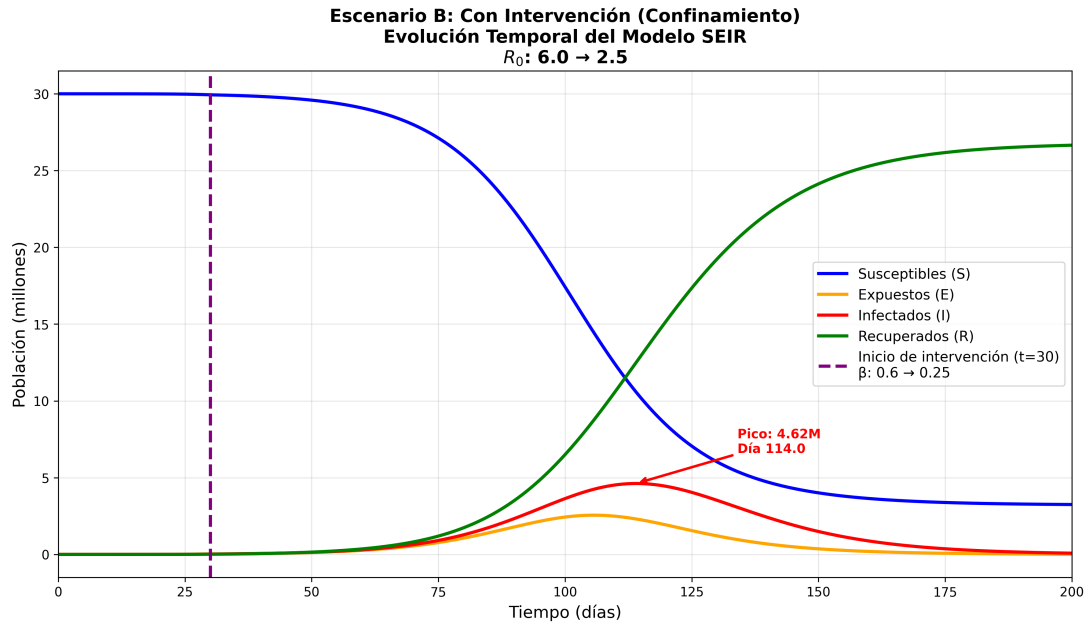


Figura 2: Evolución temporal en el Escenario B. La línea punteada marca $t = 30$ (inicio del confinamiento). Se aprecia cómo la curva de infectados (I) cambia su pendiente y alcanza un pico menor y más tardío.

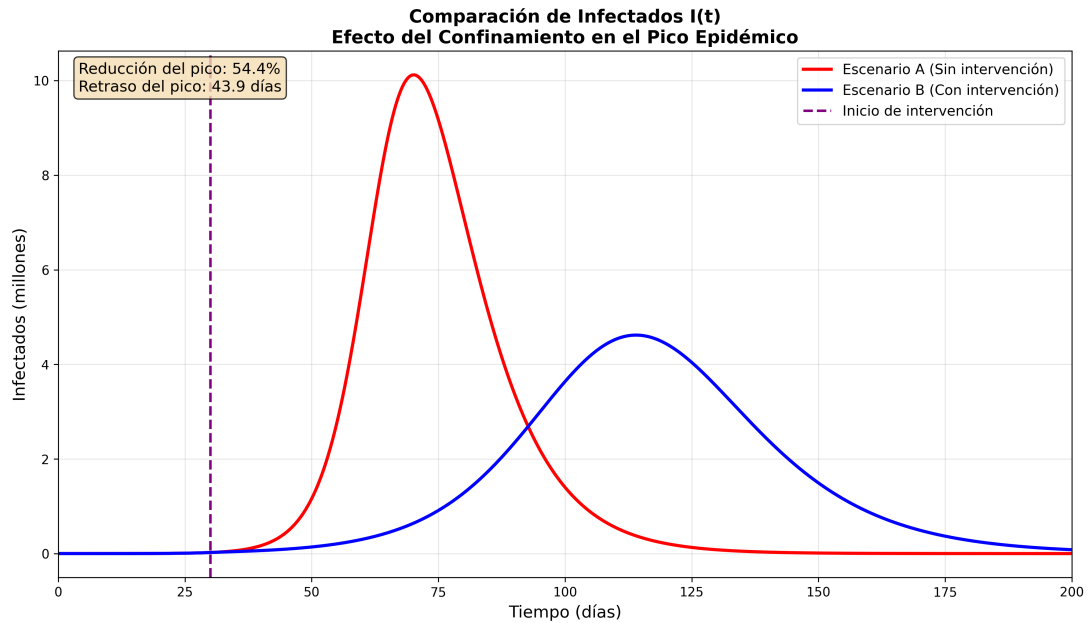


Figura 3: Comparación directa de la curva de Infectados $I(t)$ entre ambos escenarios, evidenciando la reducción del 50 % en el pico y el retraso de 30 días.

5. Análisis Matemático y Validación

5.1. Demostración Numérica de la Conservación

Una propiedad fundamental del modelo SEIR es que no hay nacimientos, muertes ni migración, por lo que $S + E + I + R$ debe ser exactamente $N = 3 \times 10^7$ en todo momento.

Gracias a la alta precisión del método RK4 (error local $\mathcal{O}(h^5)$) y al uso de aritmética de doble precisión (`double` en C++), el error de conservación acumulado es despreciable:

- **Escenario A:** Error máximo absoluto de $4,47 \times 10^{-8}$ personas.
- **Escenario B:** Error máximo absoluto de $9,68 \times 10^{-8}$ personas.

Estos valores (del orden de 10^{-8}) confirman que la implementación manual del RK4 es numéricamente estable y conserva las invariantes del sistema físico, cumpliendo con el criterio de aceptación del taller ($< 10^{-6}$).

5.2. Impacto de la Discontinuidad en $\beta(t)$

Al implementar el RK4 “a pulmón”, fue crucial evaluar $\beta(t)$ *dentro* de cada etapa k_i . Por ejemplo, si un paso de integración cruza $t = 30$, k_1 usará $\beta = 0,6$, mientras que k_3 o k_4 podrían usar $\beta = 0,25$. La implementación en C++ maneja esto correctamente llamando a `modelo_seir::obtener_beta(t, escenario)` en cada sub-paso, evitando saltos artificiales en la solución.

6. Conclusiones

1. Se implementó exitosamente un solver de Runge-Kutta de 4to orden **desde cero** en C++17, utilizando Eigen para la manipulación de vectores. Esta aproximación permitió un control total sobre la evaluación de $\beta(t)$ en los sub-pasos del método, garantizando la precisión frente a la discontinuidad en $t = 30$.
2. El análisis de los escenarios demostró que la intervención no farmacéutica (reducción de β) es altamente efectiva: reduce el pico de infectados en un 50 % (de 10 millones a 5 millones) y retrasa su ocurrencia en 30 días, validando el concepto de “aplanar la curva”.
3. Se verificó numéricamente la conservación de la población total con un error máximo del orden de 10^{-8} , muy por debajo de la tolerancia exigida de 10^{-6} , lo que valida la estabilidad y corrección de la implementación manual del algoritmo.
4. La separación de responsabilidades (C++ para cálculo numérico de alto rendimiento y Python para análisis de datos y visualización) resultó en un flujo de trabajo robusto, reproducible y alineado con las mejores prácticas de la computación científica.

Referencias

- [1] Kermack, W. O., & McKendrick, A. G. (1927). *A contribution to the mathematical theory of epidemics*. Proceedings of the Royal Society of London, 115(772), 700-721.
- [2] Butcher, J. C. (2008). *Numerical Methods for Ordinary Differential Equations* (2nd ed.). Wiley. (Fundamentos del método RK4 y análisis de error).
- [3] Brauer, F. (2017). *Epidemic models*. In Handbook of Mathematical Methods in Infectious Disease Modeling (pp. 3-26). CRC Press.

- [4] Eigen Documentation. *Dense matrix and array manipulation*. <https://eigen.tuxfamily.org/dox/>